

ข้อสอบเก่า (ระดับนัก)

คำความ

ตารางที่ 1 Truth Table

	D	C	B	A	Y
0	0	0	0	0	1
1	0	0	0	1	1
2	0	0	1	0	0
3	0	0	1	1	0
4	0	1	0	0	1
5	0	1	0	1	0
6	0	1	1	0	1
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	0
A	1	0	1	0	1
B	1	0	1	1	1
C	1	1	0	0	1
D	1	1	0	1	1
E	1	1	1	0	1
F	1	1	1	1	0

1. จาก Truth Table ในตารางที่ 1 ให้ฝึกศึกษาคำตอบคำถาม เขียนบรรยาย และวาดภาพประกอบที่เกี่ยวข้อง ดังต่อไปนี้

a) จงเขียนสมการ Boolean ให้อยู่ในรูปแบบ Product of Sum (POS) และวาดวงจร Combinational จากสมการดังกล่าวโดยใช้ Logic Gate ต่าง ๆ (3 คะแนน)

b) จงเขียนสมการ Boolean จาก Karnaugh Maps ให้อยู่ในรูปแบบ Sum of Products (SOP) (7 คะแนน) จากนั้นให้ใช้ Boolean Algebra จากนั้นให้นำสมการที่ลดรูปแล้วดังกล่าวมาเขียนเป็นวงจร Combinational โดยใช้สัญลักษณ์ Logic Gate ต่าง ๆ (3 คะแนน)

c) จงเขียนสมการ Boolean จาก 4-Way Set-Associative Memory Cache Memory ขนาด 1 Kilo word ซึ่งมี Line size ขนาด 32 ไบต์ เชื่อมต่อกับหน่วยความจำขนาด 12 bit address ซึ่งแต่ละไบต์มีขนาดเท่ากับ 8 บิต จงหาจำนวน Memory Block (2 คะแนน) จำนวน บิตของ Tag Set และ Word พร้อมวาดรูปโครงสร้างของ Main Memory Address ของ Cache นี้ (8 คะแนน) นอกจากนี้ให้วาดรูป โครงสร้างของ Memory Block (4 คะแนน) โครงสร้างของ Cache นี้ (8 คะแนน) และระบุ Memory Block ที่จะมีโอกาสอยู่ใน Set หมายเลข 3 (2 คะแนน) **รวม 4**

AND (*) OR (+)

1. 2.) Product of sum (Max Term) เลื่อน 0

1. ทั้งข้อ 1 และ ข้อ 2

$$= (D + C + \bar{B} + A)(D + C + \bar{B} + \bar{A})(D + \bar{C} + B + \bar{A})(\bar{D} + C + B + A)(\bar{D} + C + B + \bar{A})(\bar{D} + \bar{C} + \bar{B} + \bar{A})$$

LOGIC GATE SYMBOLS

ข้อ 1:

Product of Sum (Max Term) เลื่อน 0

DC \ BA	00	01	11	10
00	1	1	0	0
01	1	0	1	1
11	1	1	0	1
10	0	0	1	1

OR AND

$$= (D + C + \bar{B})(D + \bar{C} + B + \bar{A})$$

$$(\bar{D} + \bar{C} + \bar{B} + \bar{A})(\bar{D} + C + B)$$

Sum of product (Min Term) เลื่อน 1

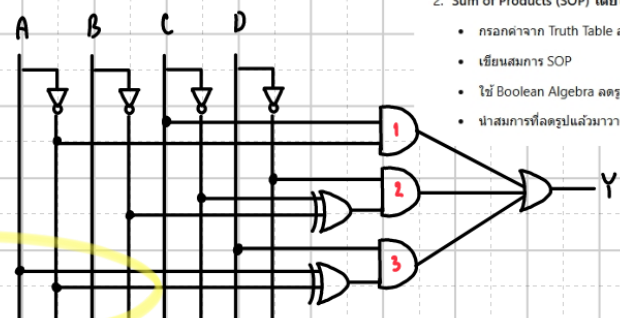
DC \ BA	00	01	11	10
00	1	1	0	0
01	1	0	1	1
11	1	1	0	1
10	0	0	1	1

$$= \bar{D}\bar{C}\bar{B} + \bar{D}C\bar{B} + C\bar{A} + D\bar{C}\bar{B} + D\bar{C}B$$

$$= C\bar{A} + \bar{D}(\bar{C}\bar{B} + C\bar{B}) + D(\bar{C}\bar{B} + C\bar{B})$$

$$= (\bar{A} + \bar{D}(\bar{C} \oplus B)) + D(A \oplus B)$$

1 2 3



- (a) แปลง Truth Table เป็นสมการ Boolean และวาดวงจร
- Product of Sum (POS)
 - เขียนสมการ POS จาก Truth Table
 - แปลงเป็นรูปแบบ Combinational Logic ด้วย AND-OR-NOT
 - Sum of Products (SOP) โดยใช้ Karnaugh Map (K-map)
 - กรอค่าจาก Truth Table ลงใน K-map
 - เขียนสมการ SOP
 - ใช้ Boolean Algebra ลดรูปสมการ
 - นำสมการที่ลดรูปแล้วมาวาดวงจร Logic Gate

2.) 4-way set Associative

ใน Cache size = 1 kilo word = $2^{10} = 1024$ words

Line (Block Size) = 32 bytes $\rightarrow$ 1 block = 32 words

Memory address = 12 bit

Word Size = 8 bit (1 word = 1 byte)

$$\text{Memory Block} = \frac{\text{Memory Address}}{\text{Line Size/block}} = \frac{2^{12}}{32/1}$$

$$= \frac{4096}{32} = 128 \text{ block}$$

$$\text{Cache Sets} = \frac{\text{Cache size}}{\text{Line Size} \times K \text{ way}} = \frac{1024}{32 \times 4} = 8 \text{ set}$$

จำนวน bit ใน Mem Add

น Tag index = $\log_2(\text{Cache Set}) = \log_2(8) = 3 \text{ bits}$

น word offset = $\log_2(\text{line size}) = \log_2(32) = 5 \text{ bit add}$

น Tag = Mem Add - (set + word) = 12 - (5 + 3) = 4 bit add

3. จงเขียนตารางอธิบายข้อดีและข้อจำกัดของเทคนิคการบริหารหน่วยความจำชนิด Partition, Paging และ Demand Paging (10 คะแนน)

บท ๑

1. Partition (การแบ่งพาร์ติชัน)

ข้อดี:

- สามารถแบ่งหน่วยความจำออกเป็นช่วงๆ ได้ไม่ปรมาณต่างๆ ใช้งานได้
- ลดการสูญเสียหน่วยความจำโดยแบ่งเป็น Fixed Partition หรือ Dynamic Partition
- มีความยืดหยุ่นในกรณีใช้ Dynamic Partition ที่สามารถปรับขนาดได้

ข้อจำกัด:

- Fixed Partition: อาจเกิด internal fragmentation (เสียพื้นที่ในพาร์ติชัน)
- Dynamic Partition: อาจเกิด external fragmentation (มีช่องว่างกระจายอยู่ทำให้ใช้หน่วยความจำไม่เต็ม)
- การจัดสรรและจัดการพาร์ติชันอาจซับซ้อน

3. Demand Paging (เพจแบบเรียกตามต้องการ)

ข้อดี:

- ลดการใช้หน่วยความจำจริง (Physical Memory) เพราะโหลดเฉพาะเพจที่ต้องใช้
- รองรับการทํางานแบบ Virtual Memory ได้ดี
- ใช้ทรัพยากรได้คุ้มค่า เพราะโหลดเฉพาะเพจที่จำเป็น

2. Paging (การแบ่งเพจ)

ข้อดี:

- ลดปัญหา external fragmentation
- ใช้ page table ในการแบ่งหน่วยความจำ ช่วยให้อุปกรณ์ใช้พื้นที่ได้อย่างมีประสิทธิภาพ
- สามารถใช้ร่วมกับ Virtual Memory ได้

ข้อจำกัด:

- เกิด internal fragmentation เพราะขนาดของ Page อาจไม่พอดีกับข้อมูล
- ต้องใช้ Page Table ซึ่งอาจทำให้เสียหน่วยความจำเพิ่ม
- มี Overhead ในการแปลง Logical Address → Physical Address

ข้อจำกัด:

- อาจเกิด Page Fault ปอบหากมีการเข้าถึงข้อมูลที่ยังไม่ถูกโหลด
- มี Overhead จากการจัดการ Page Table และ Page Replacement
- อาจเกิด Thrashing ถ้า Memory Management ไม่ดี

ข้อ 3: เปรียบเทียบเทคนิคการบริหารหน่วยความจำ

เทคนิค	ข้อดี	ข้อจำกัด
Partitioning (การแบ่งพาร์ติชัน)	- สามารถแบ่งหน่วยความจำออกเป็นช่วงๆ ได้ไม่ปรมาณต่างๆ ใช้งานได้ - ถ้าใช้ Dynamic Partitioning สามารถปรับขนาดพาร์ติชันได้ - เหมาะสำหรับระบบที่ต้องการจัดการหน่วยความจำได้อย่างไม่ปรมาณ	- Fixed Partitioning: อาจเกิด Internal Fragmentation (ช่องว่างในพาร์ติชันที่ใช้ไม่ได้) - Dynamic Partitioning: อาจเกิด External Fragmentation (ช่องว่างระหว่างพาร์ติชัน) - มีความซับซ้อนในการบริหารและการจัดการหน่วยความจำ
Paging (การแบ่งเพจ)	- ลดปัญหา External Fragmentation - โปรแกรมไม่จำเป็นต้องอยู่ในหน่วยความจำติดกัน - รองรับ Virtual Memory ทำให้อุปกรณ์ใช้หน่วยความจำได้อย่างมีประสิทธิภาพ	- อาจเกิด Internal Fragmentation เนื่องจากขนาดเพจอาจไม่พอดีกับข้อมูล - ต้องใช้ Page Table ทำให้เสียพื้นที่หน่วยความจำ - มี Overhead จากการแปลง Logical Address → Physical Address
Demand Paging (เพจแบบเรียกตามต้องการ)	- โหลดเฉพาะเพจที่ต้องใช้ ทำให้ลดการใช้หน่วยความจำจริง - ช่วยประหยัดทรัพยากรและรองรับ Virtual Memory - ใช้พื้นที่หน่วยความจำได้อย่างมีประสิทธิภาพ	- อาจเกิด Page Fault ปอบหากต้องเรียกเพจที่ยังไม่ถูกโหลด - ต้องใช้ Page Replacement Algorithm ซึ่งอาจทำให้ระบบทํางานช้าลง - หากจัดการหน่วยความจำไม่ดี อาจเกิด Thrashing (CPU ทำงานเข้าหาตัวตลิ่งไม่ลดเลย)

สรุป:

- Partitioning เหมาะกับระบบที่ต้องการแบ่งหน่วยความจำชัดเจน
- Paging ช่วยแก้ปัญหา fragmentation และรองรับ virtual memory
- Demand Paging มีประสิทธิภาพสูงแต่ต้องมีการจัดการ Page Fault ให้ดี

4. ไมโครโพรเซสเซอร์ตัวหนึ่งมี Register ขนาด 12 บิต (1 ไวร์ด) จำนวน 2 ตัวชื่อ AC และ Base โดยไมโครโพรเซสเซอร์นี้รองรับชุดคำสั่งแบบ Two-address จำนวน 8 คำสั่ง และสามารถเข้าถึงหน่วยความจำขนาด 512 ไวร์ดได้

- a) จงออกแบบคำสั่งภาษา Assembly ทั้ง 8 คำสั่ง ที่สามารถทำ Addressing Mode ชนิด Immediate, Direct, Register Indirect และ Displacement โดยวาดรูปแสดง Instruction Format (2 คะแนน), เขียนตารางแสดง List of Codes ที่ประกอบด้วย Assembly, Meaning, Operation, Addressing mode และ Opcode (Binary) (8 คะแนน)
- b) นำคำสั่งที่ออกแบบไว้มาใช้เขียนโปรแกรมเพื่อบวกเลข 2 จำนวนที่อยู่ในหน่วยความจำตำแหน่ง 100 กับ 400 และเก็บผลลัพธ์ที่ตำแหน่ง 500 หรืออธิบายการทำงานและวาดรูปประกอบ (10 คะแนน)

จัดสอบเก่า

Truth Table

	D	C	B	A	Y
0	0	0	0	0	1
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	1
5	0	1	0	1	1
6	0	1	1	0	0
7	0	1	1	1	0
8	1	0	0	0	1
9	1	0	0	1	1
A	1	0	1	0	1
B	1	0	1	1	0
C	1	1	0	0	0
D	1	1	0	1	0
E	1	1	1	0	1
F	1	1	1	1	1

1. จาก Truth Table ด้านซ้าย ให้นักศึกษาตอบคำถาม เขียนบรรยาย และวาดภาพประกอบที่เกี่ยวข้อง ดังต่อไปนี้

✓ a) จงเขียนสมการ Boolean ให้อยู่ในรูปแบบ Product of Sum (2 คะแนน) และวาดวงจร Combinational จากสมการดังกล่าวโดยใช้สัญลักษณ์ Logic Gate ต่าง ๆ (3 คะแนน)

✓ b) จงเขียนสมการ Boolean จาก Karnaugh Maps ให้อยู่ในรูปแบบ Sum of Products (7 คะแนน) จากนั้นให้ใช้ Boolean Algebra laws เพื่อลดรูปสมการให้อยู่ในรูปแบบอย่างง่ายที่สุด (5 คะแนน) จากนั้นให้นำสมการที่ลดรูปแล้วดังกล่าวมาเขียนเป็นวงจร Combinational โดยใช้สัญลักษณ์ Logic Gate ต่าง ๆ (3 คะแนน)

✓ 2. จงหาจำนวน Block (3 คะแนน), ขนาดของ Tag (3 คะแนน), Set (3 คะแนน) และ Word (3 คะแนน) ของ 2-Way Set-Associative Mapping Cache Memory ขนาด 512 ไบต์ ซึ่งมี line size ขนาด 8 ไบต์ ของหน่วยความจำขนาด 12 bit address พร้อมทั้งวาดรูปโครงสร้างของ Memory และ Cache ดังกล่าวประกอบ (8 คะแนน)

3. จงเขียนโปรแกรมที่มี Addressing Mode ชนิด Immediate

✓ Addressing, Direct Addressing, Indirect Addressing, Register

✓ Addressing และ Register Indirect Addressing อยู่ในโปรแกรมเดียวกัน พร้อมอธิบายการทำงานและวาดรูปประกอบ (20 คะแนน)

4. กำหนดให้ไมโครโพรเซสเซอร์รุ่นหนึ่งมีรีจิสเตอร์ภายในจำนวน 7 ตัวคือ Program Counter (PC), Memory Address Register (MAR), Memory Buffer Register (MBR), Input / Output Address Register (I/O AR), Input / Output Buffer Register (I/O BR), Instruction Register (IR) และ Accumulator (AC) และมีรูปแบบคำสั่งขนาด 16 บิต ตามรูปที่ 2(a) ซึ่งมี Opcode ขนาด 4 บิต และสามารถอ้างถึงตำแหน่งที่อยู่ได้ขนาด 12 บิต และใช้เทคนิคการอินเตอร์พรีต Memory mapped I/O โดย I/O เริ่มต้นที่ตำแหน่ง 0xF00h สำหรับการจับกับข้อมูลจะอยู่ในรูปแบบจำนวนเทคนิคนิยมมาตรฐาน IEEE754 Half-precision ขนาด 16 บิตตามรูปที่ 2(b) บางส่วนของค่าที่แสดงดังรูปที่ 2(c) ซึ่งตำแหน่งหน่วยความจำเริ่มต้นสำหรับโปรแกรมอยู่ที่ 0x300h และข้อมูลเริ่มต้นที่ 0xD00h

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Opcode								Address							

รูปที่ 2(a) Instruction format

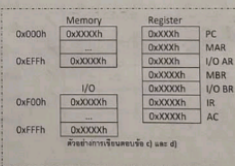
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Sign		Exponent					Mantissa								

รูปที่ 2(b) Half-precision floating-point format

Assembly	Meaning	Operation	Opcode (Binary)
LDM AC, 0x000h	Load AC from Memory	AC ← (0x000h) _{mem}	0001 1
STM AC, 0x000h	Store AC to Memory	AC → (0x000h) _{mem}	0010 2
LDIO AC, 0x000h	Load AC from I/O	AC ← (0x000h) _{i/o}	0101 5
STIO AC, 0x000h	Store AC to Memory	AC → (0x000h) _{i/o}	0110 6
ADDF AC, 0x000h	Add float to AC from Memory	AC ← AC + (0x000h) _{mem}	1001 9

รูปที่ 2(c) Partial list of opcodes

- จงแสดงวิธีการแปลงค่า 98.4 (5 คะแนน) และ -52.375 (5 คะแนน) ให้อยู่ในรูปแบบจำนวนเทคนิคนิยมมาตรฐาน IEEE754 Half-precision
- จงเขียนโปรแกรมภาษา Assembly ดังนี้
- นำค่าจาก I/O ตำแหน่ง 0xF00h (ซึ่งมีค่า 98.4) มาเก็บลงในรีจิสเตอร์ AC (1 คะแนน)
 - บวกค่าในหน่วยความจำตำแหน่ง 0xD00h (ซึ่งมีค่า -52.375) กับค่าที่อยู่ในรีจิสเตอร์ AC แล้วนำมารวมลงในรีจิสเตอร์ AC (1 คะแนน)
 - นำค่าในรีจิสเตอร์ AC เก็บลงในหน่วยความจำตำแหน่งที่ 0xD01h (1 คะแนน)
 - นำค่าในรีจิสเตอร์ AC เก็บลงที่ I/O ตำแหน่ง 0xF01h (1 คะแนน)



$$4. a) \text{ Half-precision (16 bit 1-5-10)} \Rightarrow 2^{5-1} = 16-1 = 15$$

$$98.4 \rightarrow 1100010.01100110 = 1.1000100110 \times 2^6$$

2 98	0.4 × 2 = 0.8 , 0.4 × 2 = 0.8	exponent = 15 + 6 = 21 (10101 ₂)
2 490	0.8 × 2 = 1.6 , 0.8 × 2 = 1.6	mantissa = 1000100110 ₂
2 241	0.6 × 2 = 1.2 , 0.6 × 2 = 1.2	∴ 0 10101 1000100110 ₂
2 120	0.2 × 2 = 0.4 , 0.2 × 2 = 0.4	รวม 16 → 5626
2 60		
2 30		
1 1		

$$-52.375 \rightarrow 110100.011 = 1.10100011 \times 2^5$$

2 52	0.375 × 2 = 0.75	exponent = 15 + 5 = 20 (10100 ₂)
2 260	0.75 × 2 = 1.5	mantissa = 10100011 ₂
2 130	0.5 × 2 = 1.0	∴ 1 10100 1010001100 ₂ (ยังไม่ครบ 16บิต)
2 65		รวม 16 → D29C
2 32		
1 1		

$$b.) \text{ i. LDIO AC, 0xF00h, AC} \leftarrow (0xF00h)_{I/O}$$

$$\text{ii. ADDF AC, 0xD00h, AC} \leftarrow \text{AC} + (0xD00h)_{mem}$$

$$\text{iii. STM AC, 0xD01h, AC} \rightarrow (0xD01h)_{mem}$$

$$\text{iv. STIO AC, 0xF01h, AC} \rightarrow (0xF01h)_{I/O}$$

$$(-52.375) + 98.4 = 46.025$$

↓
D29C₁₆

↓
5626₁₆

↓
51C1₁₆

$$46.025 \rightarrow 101110.000001 = 1.0111000001 \times 2^5$$

2 46	0.025 × 2 = 0.05 , 0.4 × 2 = 0.8
2 230	0.05 × 2 = 0.1 , 0.8 × 2 = 1.6
2 115	0.1 × 2 = 0.2 ,
2 57	0.2 × 2 = 0.4 ,
1 0	

$$\text{exponent} = 15 + 5 = 20 (10100_2)$$

$$\text{mantissa} = 0111000001$$

$$\therefore 0 \ 10100 \ 0111000001 \rightarrow 51C1$$

solution

0x5626 corresponds in the decimal system:
98.375

calculation path

convert to binary:

0x5626 is in binary:

0 10101 1000100110

Determine exponent:

$$\begin{aligned} 10101_2 &\triangleq 2^4 + 2^2 + 2^0 \\ &= 16 + 4 + 1 \\ &= 21 \end{aligned}$$

Subtract bias from exponent:

$$21 - 15 = 6$$

Denormalize mantissa:

Prepend '1.' to mantissas:

$$1.1000100110_2$$

The binary point must be shifted to the right by 6 digits

$$1100010.0110_2$$

Convert mantissa to decimal number:

$$\begin{aligned} 1100010.0110_2 \\ &\triangleq 2^6 + 2^5 + 2^1 + 2^{-2} + 2^{-3} \\ &= 64 + 32 + 2 + 0.25 + 0.125 \\ &= 98.375 \end{aligned}$$

c) จงเขียนรูปเพื่อแสดงคำสั่งและข้อมูลทั้งหมดที่อยู่ในตำแหน่ง 0x000h-0xFFFF ของ Memory และ I/O ก่อนโปรแกรมตามข้อ b) เริ่มทำงาน (ให้ใช้รูปแบบเลขฐาน 16 ทั้งหมด) (5 คะแนน)

-52.375 = D28C (0xD00h)

46.025 = 51C1

98.4 = 5626 (0xF00h)

C.) โครงร่างก่อนเริ่มทำงาน

Memory

0x000h	0xFFFFh
⋮	⋮
0x100h	0x5F00h
0x101h	0x9000h
0x102h	0x2D01h
0x103h	0xBF01h
⋮	⋮
⋮	⋮
0xD00h	0xD28Ch
0xD01h	0XXXXh
⋮	⋮
0xFFFFh	0XXXXh

I/O

0xF00h	0x5626h
0xF01h	0XXXXh
⋮	⋮
0xFFFFh	0XXXXh

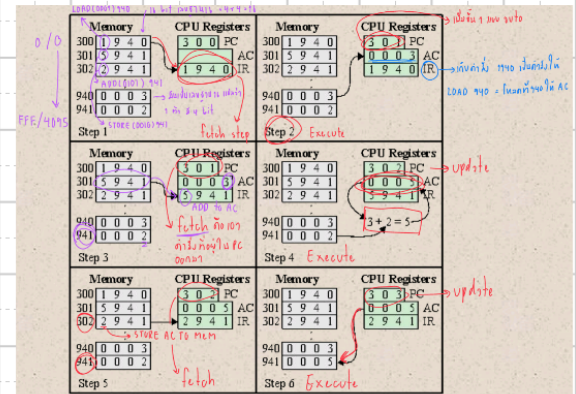
Register

0XXXXh	PC
0XXXXh	MAR
0XXXXh	I/O AR
0XXXXh	MBR
0XXXXh	I/O BR
0XXXXh	IR
0XXXXh	AC

สรุป

- ✓ MAR, MBR, I/O AR และ I/O BR เป็นหน่วยความจำชั่วคราว
- ✓ ค่าของมันจะถูกแทนที่หรือหายไปทุกครั้งที Fetch หรือ Execute คำสั่งใหม่
- ✓ เมื่อ Execute เสร็จ ค่าเดิมไม่ถูกใช้อีกและอาจกลายเป็นค่าว่างหรือถูกใช้ใหม่

PC = Program counter เก็บค่าของคำสั่งที่กำลังจะประมวลผล
IR = Instruction register เก็บคำสั่งที่นำเข้ามา
MAR = Memory address register เก็บที่เก็บค่า address
MBR = Memory buffer register → ของหน่วยความจำ
I/O AR = Input/output address register → เก็บค่า MAR กับ MBR แต่ให้เก็บ 3/6
I/O BR = Input/output buffer register



d) จงเขียนรูปเพื่อแสดงค่าใน Memory, I/O, Register AC, PC และ IR ที่เกิดขึ้นหลังกระบวนการ Fetch และ Execute ของโปรแกรมตามข้อ b) พร้อมเขียนคำอธิบายเหตุการณ์ที่เกิดขึ้นในแต่ละคำสั่ง (ให้ใช้รูปแบบเลขฐาน 16 ทั้งหมด) (16 คะแนน)

e) ผลลัพธ์ของโปรแกรมนี้ที่อยู่ในรีจิสเตอร์ AC มีค่าเท่าไร? (ให้ตอบโดยใช้รูปแบบเลขฐาน 16) (5 คะแนน)

Memory

0x000h	0xFFFFh
⋮	⋮
0x100h	0x5F00h
0x101h	0x9000h
0x102h	0x2D01h
0x103h	0xBF01h
⋮	⋮
⋮	⋮
0xD00h	0xD28Ch
0xD01h	0XXXXh
⋮	⋮
0xFFFFh	0XXXXh

I/O

0xF00h	0x5626h
0xF01h	0XXXXh
⋮	⋮
0xFFFFh	0XXXXh

Register

0x100h	PC
0x100h	MAR
0XXXXh	I/O AR
0x5F00h	MBR
0XXXXh	I/O BR
0x5F00h	IR
0XXXXh	AC

คำสั่งที่ i Fetch
step 1

LDIO AC, 0xF00h

ดึงค่าส่งมาจาก RAM 100h

Memory

0x000h	0xFFFFh
⋮	⋮
0x100h	0x5F00h
0x101h	0x9000h
0x102h	0x2D01h
0x103h	0xBF01h
⋮	⋮
⋮	⋮
0xD00h	0xD28Ch
0xD01h	0XXXXh
⋮	⋮
0xFFFFh	0XXXXh

I/O

0xF00h	0x5626h
0xF01h	0XXXXh
⋮	⋮
0xFFFFh	0XXXXh

Register

0x101h	PC
0XXXXh	MAR
0x5F00h	I/O AR
0XXXXh	MBR
0x5626h	I/O BR
0x5F00h	IR
0x5626h	AC

คำสั่งที่ i Execute
step 2

รับค่าส่ง I/O

Memory

0x000h	0XXXXXh
...	...
0x100h	0x5F00h
0x101h	0x9D00h
0x102h	0x2D01h
0x103h	0x6F01h
...	...
0xD00h	0xD24Ch
0xD01h	0XXXXXh
...	...
0xEFFh	0XXXXXh

I/O

0xF00h	0x5126h
0xF01h	0XXXXXh
...	...
0xFFh	0XXXXXh

Register

0x101h	PC
0x101h	MAR
0XXXXXh	I/O AR
0x9D00h	MBR
0XXXXXh	I/O BR
0x9D00h	IR
0x5126h	AC

คำสั่ง ii Fetch
step 3

ADDF AC, 0xD00h

Memory

0x000h	0XXXXXh
...	...
0x100h	0x5F00h
0x101h	0x9D00h
0x102h	0x2D01h
0x103h	0x6F01h
...	...
0xD00h	0xD24Ch
0xD01h	0XXXXXh
...	...
0xEFFh	0XXXXXh

I/O

0xF00h	0x5126h
0xF01h	0XXXXXh
...	...
0xFFh	0XXXXXh

Register

0x102h	PC
0x9D06h	MAR
0XXXXXh	I/O AR
0xD24Ch	MBR
0XXXXXh	I/O BR
0x9D00h	IR
0x51C1h	AC

คำสั่ง ii Execute
step 4

$$-52.575 + 98.4 = 46.025$$

Memory

0x000h	0XXXXXh
...	...
0x100h	0x5F00h
0x101h	0x9D00h
0x102h	0x2D01h
0x103h	0x6F01h
...	...
0xD00h	0xD24Ch
0xD01h	0XXXXXh
...	...
0xEFFh	0XXXXXh

I/O

0xF00h	0x5126h
0xF01h	0XXXXXh
...	...
0xFFh	0XXXXXh

Register

0x102h	PC
0x102h	MAR
0XXXXXh	I/O AR
0x2D01h	MBR
0XXXXXh	I/O BR
0x2D01h	IR
0x51C1h	AC

คำสั่ง iii Fetch
step 5

STM AC, 0xD01h

Memory

0x000h	0XXXXXh
...	...
0x100h	0x5F00h
0x101h	0x9D00h
0x102h	0x2D01h
0x103h	0x6F01h
...	...
0xD00h	0xD24Ch
0xD01h	0x51C1h
...	...
0xEFFh	0XXXXXh

I/O

0xF00h	0x5126h
0xF01h	0XXXXXh
...	...
0xFFh	0XXXXXh

Register

0x103h	PC
0x2D01h	MAR
0XXXXXh	I/O AR
0x51C1h	MBR
0XXXXXh	I/O BR
0xD201h	IR
0x51C1h	AC

คำสั่ง iii Execute
step 6

Memory

0x000h	0XXXXXh
⋮	⋮
0x100h	0x5F00h
0x101h	0x9000h
0x102h	0x2D01h
0x103h	0x6F01h
⋮	⋮
⋮	⋮
0xD00h	0xD28Ch
0xD01h	0x51C1h
⋮	⋮
0xEFFh	0XXXXXh

I/O

0xF00h	0x5126h
0xF01h	0XXXXXh
⋮	⋮
0xFFFh	0XXXXXh

Register

0x103h	PC
0x103h	MAR
0XXXXh	I/O AR
0x6F01h	MBR
0XXXXh	I/O DR
0x1F01h	IR
0x51C1h	AC

คำสั่ง iv Fetch

step 7

STIO AC, 0xF01h

Memory

0x000h	0XXXXXh
⋮	⋮
0x100h	0x5F00h
0x101h	0x9000h
0x102h	0x2D01h
0x103h	0x6F01h
⋮	⋮
⋮	⋮
0xD00h	0xD28Ch
0xD01h	0x51C1h
⋮	⋮
0xEFFh	0XXXXXh

I/O

0xF00h	0x5126h
0xF01h	0x51C1h
⋮	⋮
0xFFFh	0XXXXXh

Register

0x104h	PC
0XXXXh	MAR
0x6F01h	I/O AR
0XXXXh	MBR
0x51C1h	I/O DR
0x6F01h	IR
0x51C1h	AC

คำสั่ง iv Execute

step 8

⑤ - 48.8 1101102 1101102 1101102

$$48 = 11000_2$$

$$0.8 = 100110$$

$$\begin{array}{r} 2 \cdot 48 - 48 = 0 \\ 2 \cdot 24 - 24 = 0 \\ 2 \cdot 12 - 12 = 0 \\ 2 \cdot 6 - 6 = 0 \\ 2 \cdot 3 - 2 = 1 \\ 1 \end{array}$$

$$\begin{array}{l} 2 \times 0.8 = 1.6 \\ 2 \times 0.6 = 1.2 \\ 2 \times 0.2 = 0.4 \\ 2 \times 0.4 = 0.8 \\ 2 \times 0.8 = 1.6 \end{array}$$

$$= 110000_2$$

$$= 11.00$$

$$110000.1100 = 1.100001100 \times 2^5$$

$$E = 15 + 5 = 20 = 10100$$

$$S = 1$$

$$E = 10100$$

$$M = 1000011000$$

S	E	M
1	10100	1000011000
D	2	1 8

$$HEX = D218$$

$$1101102$$

$$10100 = 20$$

$$20 - 15 = 5$$

$$1.100000.11000 = 1100000.11000$$

$$= 48.75$$

$$75.4$$

$$75 = 1001011_2$$

$$0.4 = 0110011_2$$

$$1001011.011011_2 = 1.001011011011 \times 2^6$$

$$E = 15 + 6 = 21 = 10101$$

$$S = 0$$

$$E = 10101$$

$$M = 0010110110$$

S	E	M
0	10101	0010110110
5	4	B 6

$$54B6$$

$$1101102$$

$$10101 = 21$$

$$21 - 15 = 6$$

$$1.0010110110 = 1001011.0110$$

$$= 75.375$$

$$26.625$$

$$2 \times 0.625 = 1.25$$

$$2 \times 0.25 = 0.5$$

$$2 \times 0.5 = 1.0$$

$$2 \times 0 = 0$$

$$26 = 11010$$

$$0.625 = 101$$

$$11010.101 = 1.1010101 \times 2^4$$

$$E = 15 + 4 = 19$$

$$= 10011$$

$$S = 0$$

$$E = 1011$$

$$M = 1010101000$$

S	E	M
0	1011	1010101000
5	A	A 8

$$5A A 8$$

$$1101102$$

$$10110 = 19$$

$$19 - 15 = 4$$

$$1.1010101000 = 11010.1010$$

$$= 26.625$$

i) นำค่าในหน่วยความจำที่ตำแหน่ง $0 \times E00h$ (-48.9) มาใส่ใน 3 บิตของ AC

LDM AC, $0 \times E00h$, $AC \leftarrow (0 \times E00h)_{mem}$

ii) นำค่าใน 3 บิตของ AC เก็บลงใน I/O ที่ตำแหน่ง $0 \times FFFh$

STIO AC, $0 \times FFFh$, $AC \rightarrow (0 \times FFFh)_{i/o}$

iii) นำค่าในหน่วยความจำที่ตำแหน่ง $0 \times E01h$ (75.4) มาบวกกับค่าที่อยู่ใน 3 บิตของ AC แล้วนำผลลัพธ์มาใส่ใน 3 บิตของ AC

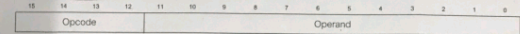
ADD AC, $0 \times E01h$, $AC \leftarrow AC + (E01h)_{mem}$

iv) นำค่าใน 3 บิตของ AC เก็บลงในหน่วยความจำที่ตำแหน่ง $0 \times E02h$

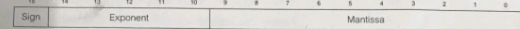
STM AC, $0 \times E02h$, $AC \rightarrow (0 \times E02h)_{mem}$

ชื่อ-นามสกุล: นาย ดิวิษฐ์ ก้อนแก้ว รหัสประจำตัวนักศึกษา: 6509000829

5. กำหนดให้ไมโครโพรเซสเซอร์รุ่นหนึ่งมีรีจิสเตอร์ภายในจำนวน 3 ตัวคือ Program Counter (PC), Instruction Register (IR) และ Accumulator (AC) และมีรูปแบบคำสั่งขนาด 16 บิตตามรูปที่ 1 ซึ่งมี Opcode ขนาด 4 บิต และสามารถอ้างถึงตำแหน่งที่อยู่ในขนาด 12 บิต และให้เทคนิคการอินเตอร์พรีตชนิด Memory mapped I/O โดย I/O เริ่มต้นที่ตำแหน่ง $0 \times F00h$ สำหรับการจับเก็บข้อมูลจะอยู่ในรูปแบบจำนวนทศนิยมมาตรฐาน IEEE754 Half-precision ขนาด 16 บิตตามรูปที่ 2 บางส่วนของคำสั่งแสดงดังรูปที่ 3 ซึ่งตำแหน่งหน่วยความจำเริ่มต้นสำหรับโปรแกรมอยู่ที่ $0 \times 100h$ และข้อมูลเริ่มต้นที่ $0 \times E00h$



รูปที่ 1 Instruction format



รูปที่ 2 Half-precision floating-point format

Assembly	Meaning	Operation	Opcode (Binary)
LDM AC, $0 \times 000h$	Load AC from Memory	$AC \leftarrow (0 \times 000h)_{mem}$	1000 0
STM AC, $0 \times 000h$	Store AC to Memory	$AC \rightarrow (0 \times 000h)_{mem}$	0000 0
LDIO AC, $0 \times 000h$	Load AC from I/O	$AC \leftarrow (0 \times 000h)_{i/o}$	1001 0
STIO AC, $0 \times 000h$	Store AC to I/O	$AC \rightarrow (0 \times 000h)_{i/o}$	0001 0
ADDF AC, $0 \times 000h$	Add AC with float from Memory to AC	$AC \leftarrow AC + (0 \times 000h)_{mem}$	0101 5

รูปที่ 3 Partial list of codes

- จงแสดงวิธีการแปลงค่า -48.8 (5 คะแนน) และ 75.4 (5 คะแนน) ให้อยู่ในรูปแบบจำนวนทศนิยมมาตรฐาน IEEE754 Half-precision
- จงเขียนโปรแกรมภาษา Assembly ต่อไปนี้
 - นำค่าจากหน่วยความจำตำแหน่ง $0 \times E00h$ (ซึ่งมีค่า -48.8) มาเก็บลงในรีจิสเตอร์ AC (1 คะแนน)
 - นำค่าในรีจิสเตอร์ AC เก็บลงใน I/O ตำแหน่ง $0 \times FFFh$ (1 คะแนน)
 - นำค่าในหน่วยความจำตำแหน่ง $0 \times E01h$ (ซึ่งมีค่า 75.4) มาบวกกับค่าที่อยู่ในรีจิสเตอร์ AC แล้วนำมาเก็บลงในรีจิสเตอร์ AC (1 คะแนน)
 - นำค่าในรีจิสเตอร์ AC เก็บลงในหน่วยความจำตำแหน่งที่ $0 \times E02h$ (1 คะแนน)
- จงเขียนรูปเพื่อแสดงค่า (เฉพาะตำแหน่งที่มีคำสั่งหรือข้อมูล) ใน Memory, I/O, Register ที่เกิดขึ้นก่อนโปรแกรมตามข้อ b) เริ่มต้นทำงาน โดยใช้ตัวอย่างตามรูปที่ 4 (ให้ใช้รูปแบบเลขฐาน 16 ทั้งหมด) (3 คะแนน)
- จงเขียนรูปเพื่อแสดงค่า (เฉพาะตำแหน่งที่มีคำสั่งหรือข้อมูล) ใน Memory, I/O, Register ที่เกิดขึ้นหลังการประมวลผล Fetch และ Execute คำสั่งที่ 3 ของโปรแกรมตามข้อ b) พร้อมเขียนคำอธิบายเหตุการณ์ที่เกิดขึ้น (ให้ใช้รูปแบบเลขฐาน 16 ทั้งหมด) (11 คะแนน)
- ผลลัพธ์ของโปรแกรมนี้จะอยู่ในรีจิสเตอร์ AC มีค่าเท่าไร (ให้ตอบโดยใช้รูปแบบเลขฐาน 10) (2 คะแนน)

1001011

- c) จงเขียนรูปเพื่อแสดงค่า (เฉพาะตำแหน่งที่มีคำสั่งหรือข้อมูล) ใน Memory, I/O, Register ที่เกิดขึ้นก่อนโปรแกรมตามข้อ b) เริ่มทำงาน โดยใช้ตัวอย่างตามรูปที่ 4 (ให้ใช้รูปแบบเลขฐาน 16 ทั้งหมด) (3 คะแนน)

$$D218 = -48.75$$

$$54B6 = 75.375$$

$$5AA8 = 26.625$$

Memory

0x000h	0xxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0xxxxxh

I/O

0xFFfh	0xxxxxh
--------	---------

Register

0xxxxh	PC
0xxxxh	IR
0xxxxh	AC

Memory

0x000h	0xxxxxh
:	.
0x100h	0x
0x101h	0x
0x102h	0x
0x103h	0x
:	.
0xE00h	0x
0xE01h	0x
0xE02h	0x

I/O

0xFFfh	0x
--------	----

Register

0x	h	PC
0x	h	IR
0x	h	AC

Memory

0x000h	0xxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

Register

0x100h	PC
0x8E00h	IR
0xxxxxh	AC

aishin (i)

step 1 Fetch
LDM AC, 0xE00h

I/O

0xFFh 0xD218h

Memory

0x000h	0xxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

Register

0x101h	PC
0x8E00h	IR
0xD218h	AC

aishin (i)

step 2 Execute

I/O

0xFFh 0xD218h

Memory

0x000h	0xxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

Register

0x101h	PC
0x1FFFh	IR
0xD218h	AC

aishin (ii)

step 3 Fetch
STIO AC, 0xFFh

I/O

0xFFh 0xD218h

Memory

0x000h	0xxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

Register

0x102h	PC
0x1FFFh	IR
0xD218h	AC

aishin ii

step 4
Execute

I/O

0xFFh 0xD218h

d

Memory

0x000h	0xxxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

I/O

0xFFh 0xD218h

Register

0x102h	PC
0x5E01h	IR
0xD218h	AC

0121h (iii)

step 5 Fetch

ADD AC, AC + (0xE01h)
mem

Memory

0x000h	0xxxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

I/O

0xFFh 0xD218h

Register

0x103h	PC
0x0E02h	IR
0x5AA8h	AC

0121h (iv)

step 7 Fetch

STM AC, 0xE02h

Memory

0x000h	0xxxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

I/O

0xFFh 0xD218h

Memory

0x000h	0xxxxxxh
:	.
0x100h	0x8E00h
0x101h	0x1FFFh
0x102h	0x5E01h
0x103h	0x0E02h
:	.
0xE00h	0xD218h
0xE01h	0x54B6h
0xE02h	0x5AA8h

I/O

0xFFh 0xD218h

Register

0x103h	PC
0x5E01h	IR
0x5AA8h	AC

0121h (iii)

step 6 Execute

 $(-40.75) + 75.975 = 26.625$

(e)

Register

0x104h	PC
0x0E02h	IR
0x5AA8h	AC

step 8 Execute

no. way

4 way Set - Associative

Cache Memory 1 kilo word = 1024 byte

Line size 32 Byte

Memory word 12 bit address

1 bit = 0,1

1 byte = 8 bit

1 kilobyte = 1024 byte

1 word = 8 bit

1 word = 1 byte

$$\text{Memory Block} = \frac{\text{Memory Address}}{\text{Line size/block}}$$

$$= \frac{2^{12}}{32/1}$$

$$= \frac{4096}{32}$$

$$= 128 \text{ block}$$

Tag 4 bit Address

set 3 bit Address

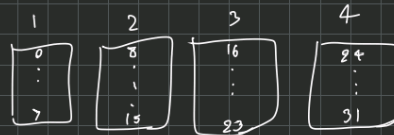
word 5 bit Address

$$\text{no Set} = \frac{\text{Cache Memory}}{\text{Line size} \times \text{way}}$$

$$= \frac{1024 \text{ byte}}{32 \times 4}$$

$$= \frac{8}{1}$$

$$= 1 \text{ set } 8 \text{ byte}$$



no set bit Address

$$= \log_2(8)$$

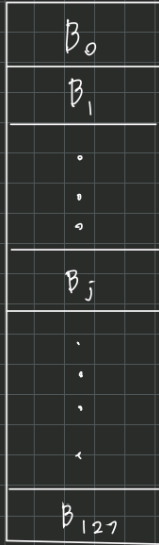
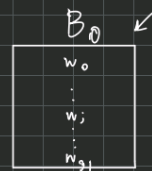
$$= \log_2(8)$$

$$= 2^3$$

$$= 3$$

Memory = 2^{12}

= 4096 byte



$$w = \log_2(\text{line size})$$

$$= \log_2(32)$$

$$= 2^5$$

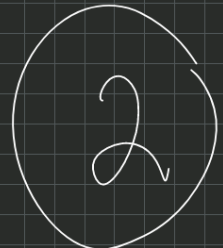
$$= 5 \text{ bit Address}$$

Tag = word bit Address in row

= number of Set + Word
address bit Address

$$= 12 - (3 + 5)$$

$$\text{Tag} = 4$$



Memory Address

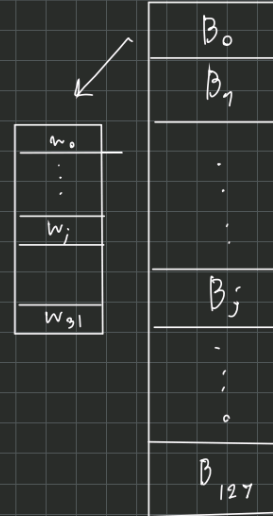
Tag	set	word
4 bits	5 bits	3 bits

Cache

HEX	Tag	Data	Set Number	Tag	Data
000	F ₀		0000		
⋮	⋮		0001		
00P	F _j		⋮		
⋮	⋮		⋮		
01F	F ₇		0007		

Tag	Data	Set Number	Tag	Data
		0000		
		0001		
		⋮		
		⋮		
		0007		

Main Memory



2